

Visvesvaraya Technological University, Belagavi – 590018



PROJECT REPORT
ON
**Sand Boil Detection and Prediction: A Visual
Approach**

Submitted in partial fulfillment for the award of degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE & ENGINEERING**

Submitted by

Sharanya S Shetty	4SO20CS141
Tina Benita Rego	4SO20CS171
Vaishnavi Bharadwaj B	4SO20CS173
Vanesha Relitsa Cutinho	4SO20CS174

Under the Guidance of

Dr Sunitha G

Associate Professor, Department of CSE



**DEPT. OF COMPUTER SCIENCE AND ENGINEERING
ST JOSEPH ENGINEERING COLLEGE
An Autonomous Institution**

(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)

Vamanjoor, Mangaluru - 575028, Karnataka

2023-24

ST JOSEPH ENGINEERING COLLEGE

An Autonomous Institution

(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)

Vamanjoor, Mangaluru - 575028, Karnataka

DEPT. OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the project work entitled "Sand Boil Detection and Prediction: A Visual Approach" carried out by

Sharanya S Shetty	4SO20CS141
Tina Benita Rego	4SO20CS171
Vaishnavi Bharadwaj B	4SO20CS173
Vanesha Relitsa Cutinho	4SO20CS174

the bonafide students of VIII semester Computer Science & Engineering in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2023-2024. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Dr Sunitha G
Project Guide

Dr Sridevi Saralaya
HOD-CSE

Dr Rio D'Souza
Principal

External Viva:

Examiner's Name

Signature with Date

1.

.....

2.

.....

ST JOSEPH ENGINEERING COLLEGE

An Autonomous Institution

(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)

Vamanjoor, Mangaluru - 575028, Karnataka

DEPT. OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We hereby declare that the entire work embodied in this Project Report titled "**Sand Boil Detection and Prediction: A Visual Approach**" has been carried out by us at St Joseph Engineering College, Mangaluru under the supervision of **Dr Sunitha G**, for the award of **Bachelor of Engineering in Computer Science & Engineering**. This report has not been submitted to this or any other University for the award of any other degree.

Sharanya S Shetty	4SO20CS141
Tina Benita Rego	4SO20CS171
Vaishnavi Bharadwaj B	4SO20CS173
Vanesha Relitsa Cutinho	4SO20CS174

Acknowledgement

We dedicate this page to acknowledge and thank those responsible for the shaping of the project. Without their guidance and help, the experience while constructing the dissertation would not have been so smooth and efficient.

We sincerely thank our Project guide **Dr Sunitha G**, Associate Professor, Computer Science and Engineering for his guidance and valuable suggestions which helped us to complete this project. We also thank our Project coordinator **Dr Supriya Salian**, Dept of CSE, for his consistent encouragement.

We owe our profound gratitude to **Dr Sridevi Saralaya**, Head of the Department, Computer Science and Engineering, whose kind support and guidance helped us to complete this work successfully.

We are extremely thankful to our Principal, **Dr Rio D'Souza**, Director, Rev. **Fr Wilfred Prakash D'Souza**, and Assistant Director, **Rev Fr Kenneth Rayner Crasta** for their support and encouragement.

We would like to thank all faculty and staff of the Department of Computer Science and Engineering who have always been with us extending their support, precious suggestions, guidance, and encouragement through the project.

We also extend our gratitude to our friends and family members for their continuous support.

Abstract

Sandboils play a crucial role in geotechnical engineering and disaster management. They emerge during periods of heightened groundwater pressure triggered by events like earthquakes or heavy rainfall, posing risks such as soil liquefaction and infrastructure instability. Detecting sandboils is pivotal for minimizing their impact and bolstering disaster readiness.

Literature reveals ongoing efforts in sandboil detection, emphasizing the need for efficient methodologies. This project proposes a novel approach involving image acquisition of affected areas, followed by image processing algorithms to identify sandboils.

The research methodology entails experimentation with image datasets from known sandboil-prone regions. Results confirm the effectiveness of the proposed method, offering automated sandboil detection that expedites response times and enhances risk management strategies.

The automated image processing solution reduces reliance on manual inspections and enabling swift responses to potential threats. This translates to cost savings, improved labor efficiency, and heightened overall disaster preparedness.

Future endeavors could focus on algorithm refinement, integration of additional sensor data, and usage of drones, expanding the scope of application and enhancing disaster management capabilities.

Table of Contents

Acknowledgement	i
Abstract	ii
Table of Contents	v
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	2
1.3 Scope	2
2 Literature Survey	4
2.1 Significance of Sand Boil Detection Near Levees [1]	4
2.2 Experimental study on the mechanism of sand boils and associated settlements due to soil liquefaction in loose sand [2]	5
2.3 Evaluation of Settlements in Sand Deposits Following Liquefaction During Earthquakes [3]	6
2.4 Effects of porous media thickness and its hydraulic gradient history on the formation of sand boils: Experimental investigation [4]	7
2.5 Observations on SandBoils from Simple Model Tests [5] . .	8
2.6 Object Detection [6]	10

2.7	Machine Learning Methods [7]	11
2.8	Viola-Jones' Object Detector [8]	15
2.9	You Only Look Once (YOLO) Object Detector [9]	16
2.10	Single Shot Multibox Detector [10]	17
2.11	Stacking [11]	18
2.12	Summary	19
2.13	Proposed System	23
2.13.1	Why the chosen problem / project is important? . .	23
2.13.2	What is novel in proposed project work?	23
2.13.3	How would it advance the state-of-the-art?	23
2.13.4	How does your tool/technique/approach differ from each of the existing works that you have surveyed?	24
3	Methodology	25
3.1	Algorithms	25
3.1.1	Viola Jones	25
3.1.2	You Only Look Once (YOLO)	26
3.1.3	Support Vector Machine (SVM)	28
3.1.4	Convolution Neural Network (CNN)	29
3.2	Design	31
4	Implementation	35
4.1	Description of Dataset	35
4.2	Features of the Dataset	36
4.3	Experimental Setup	41
4.3.1	Feature Extraction	41
4.3.2	Haralick Features	41
4.3.3	Hu Moments	42
4.3.4	Histograms	42
4.3.5	Histogram of Oriented Gradients (HOG)	43
4.4	Hardware and Software	45
4.4.1	Software Requirements	45
4.4.2	Hardware Requirements	45

5	Results and Discussion	46
5.1	Viola Jones	46
5.2	YOLO	48
5.3	SVM	52
5.4	CNN	53
5.5	Comparison Table	54
6	Conclusions and Future Work	55
	References	56

List of Figures

2.1	Sandboil Formation	5
2.2	Illustration of sand boils using before and after images of test DM1	9
2.3	Illustration of “silt clouds” from tests a) DM2; b) DM3. . .	9
2.4	Difference between SSD and YOLO methods	12
3.1	Viola-Jones’ Algorithm	26
3.2	YOLO Algorithm	27
3.3	SVM Algorithm	29
3.4	CNN Algorithm	30
3.5	Architecture Diagram	33
4.1	Positive Image	37
4.2	Positive Image	37
4.3	Positive Image	38
4.4	Positive Image	38
4.5	Negative Image	39
4.6	Negative Image	39
4.7	Negative Image	40
4.8	Negative Image	40
4.9	Feature Extraction	44
5.1	Viola Jones Algorithm Detecting a Sand Boil	46
5.2	Viola Jones Algorithm Detecting a Sand Boil	47
5.3	YOLO Algorithm Training Images	48
5.4	YOLO Algorithm Testing Images	49
5.5	YOLO Algorithm Prediction	50

5.6	YOLO Algorithm Graphs	51
5.7	SVM Algorithm Detecting Sand Boil	52
5.8	CNN Detecting Sand boil	53

List of Tables

2.1	Accuracies of Different Methods	20
2.2	Comparison of different methods	21
5.1	Comparison of all the Algorithms and the Datasets	54

Chapter 1

Introduction

1.1 Background

In article [1] The detection of sandboils is a critical aspect of geotechnical and geological hazard assessment, particularly in regions prone to flooding, seismic activity, or areas with high groundwater levels. Sandboils, also referred to as sand boils or sand boils, are geological phenomena that occur when water under pressure forces its way to the surface, carrying fine sand and silt with it. This phenomenon is commonly associated with events such as floods, earthquakes, or other disturbances that increase water pressure in the ground.

The key steps of the image processing approach include pre-processing, feature extraction, and classification. In the pre-processing stage, the acquired images are enhanced and filtered to improve the quality of relevant features. Feature extraction involves identifying distinctive patterns and attributes that are indicative of sandboils. Finally, a classification algorithm is employed to differentiate between areas with sandboils and those without.

The background of the proposed visual approach using image processing lies in the advancement of technology and the increasing availability of high-resolution imaging devices. Leveraging these technologies for automated sandboil detection can significantly improve the speed and accuracy of hazard assessment, leading to more effective disaster preparedness and response strategies. This research contributes to the ongoing efforts to de-

velop innovative and technology-driven solutions for geohazard detection and management.

1.2 Problem statement

The issue at hand is the lack of a reliable and timely detection mechanism for sand boils, a geotechnical hazard with potentially disastrous effects on infrastructure and public safety. Traditional detection methods are time-consuming, expensive, and frequently incapable of providing early indications. The main aim of our work is to address this significant issue by developing an automated detection system and predictive model using image processing technique. The method will greatly improve the ability to detect sand boils quickly, allowing for proactive action and hazard mitigation.

1.3 Scope

The project's importance in the actual world is considerable since it addresses a serious issue with broad applications across multiple fields. The following are the areas of application where the project can have a significant impact:

Geotechnical Engineering and Infrastructure Management: The initiative has the potential to transform how geotechnical engineers monitor soil stability and decrease the risk of sand boils, ultimately improving the resilience of vital infrastructure such as highways, bridges, and levees.

Natural Disaster Management: Sandboils are frequently caused by natural disasters such as earthquakes, floods, and excessive rainfall. The early warning system developed for the project can help disaster management authorities make timely choices to protect communities and assets.

Planning for Construction and Development: This device can help the construction sector by recognizing possible sand boil concerns before starting projects, resulting in safer construction practices and cost savings.

Land Use Planning and Agriculture: Sandboils have the potential to im-

pair agricultural activity and land use. The study can help identify and manage sandboil-prone locations, allowing for better land-use decisions.

Civil Engineering Study: This project's data can contribute to current civil engineering research, allowing for a better understanding of sand boil production and its causes.

Management of Water Resources: Sandboils can have an impact on water quality and flow patterns. The project can help monitor and manage water resources in sandboil-prone areas.

In summary, the project's importance spans numerous disciplines, including engineering and disaster management, as well as environmental monitoring and public safety. Its possible applications could save lives, safeguard infrastructure, and improve the overall resilience of populations and ecosystems in sandboil-prone locations.

Chapter 2

Literature Survey

This chapter gives the details of the various works which were carried out for sandboil detection.

2.1 Significance of Sand Boil Detection Near Levees [1]

In [1] Levees are embankments or flood banks constructed along naturally occurring water bodies in order to stop flooding . They provide protection for vast amounts of commercial and residential properties, especially in the New Orleans area. However, levees and flood-walls degrade over time due to the impact of severe weather, development of sand boils, subsidence of land, seepage, development of cracks, etc. Further, published data indicates that coastal Louisiana lost approximately 16 square miles of land between 1985 and 2010. In 2005, there were over 50 failures of the levees and flood walls protecting New Orleans, Louisiana, and its surrounding suburbs following the passage of Hurricane Katrina and landfall in Mississippi. These failures caused flooding in 80% of the city of New Orleans and all of St. Bernard Parish. Events like Hurricane Katrina in 2005 have shown that levee failures can be catastrophic, and very costly. If these levees fail due to these conditions, there is a potential for significant property damage, and loss of life, as experienced in New Orleans in the 2005 hurricane, Katrina. It is of great importance to monitor the physical conditions of levees for flood control .

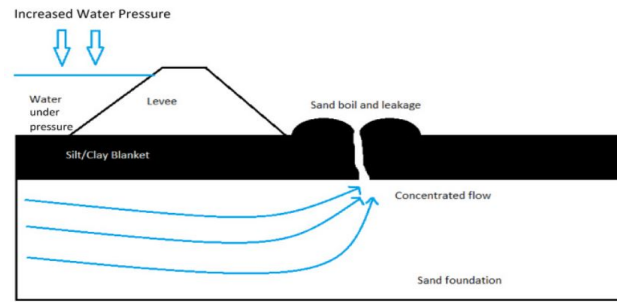


Figure 2.1: Sandboil Formation

Since sand boils have some characteristic features such as being circular, and with a discernible center area, it is a good subject for a machine learning model to be able to make accurate predictions on. This fault or deficiency, shown in Fig. 2.1, usually appears as bubbling or flowing orifice on the land side of a levee. They are typically considered a substantial hazard, especially if they are large and/or have moving soil in [1]. Observations such as the throat width (width of the opening) and the height and diameter if a cone of sand is formed around it must be measured to determine the severity of the sand boil. Levees need to be appropriately maintained and actively monitored for failure. Due to the tremendous length and variations of levee structures, proper operation and maintenance of levees can be challenging, especially when it is currently done manually using physical surveys which are a drain on time and resources. This thesis aims to speed up this process and assist in the monitoring process of levees and coastal changes by detecting sand boil from images, expected to be collected by drones for monitoring the levees.

2.2 Experimental study on the mechanism of sand boils and associated settlements due to soil liquefaction in loose sand [2]

In [2] The mechanism of sand boiling and its induced settlements were investigated with model testing. The thicknesses of the non-liquefiable layer, the gap sizes, and the overburden pressures were varied in the test program composed of 16 tests. On the basis of the PWP transducers

recording transient variations of the water pressure, one laser displacement sensor measuring the plate settlement, and high-speed camera monitoring the process, the mechanism of sand boiling was discussed insightfully.

2.3 Evaluation of Settlements in Sand Deposits Following Liquefaction During Earthquakes [3]

In [3] By examining a bulk of laboratory test data on sands obtained by using a simple shear apparatus, a family of curves was established in which the volumetric strain resulting from dissipation of pore water pressures is correlated with the density of sand and conventionally used factor of safety against liquefaction. Thus, given the factor of safety and the density in each layer of a sand deposit at a given site, the volumetric strain can be calculated and by integrating the volume changes throughout the depth, it becomes possible to estimate the amount of settlements on the ground surface produced by shaking during earthquakes. The outcome of the data arrangements as above was put in a framework of methodology to estimate the liquefaction-induced settlements of the ground. The proposed methodology was used to estimate the settlements at several sites devastated by liquefaction during the 1964 Niigata earthquake. The estimated values of the settlements were examined in the light of known performances of the sand deposits at respective site. It was shown that the proposed methodology may be used for predicting liquefaction-induced settlements with a level of accuracy suitable for many engineering purposes.

When saturated sand deposits are subjected to shaking during an earthquake, pore water pressures are known to build up leading to liquefaction or loss of strength in the sand deposits. The pore water pressures then start to dissipate mainly towards the ground surface, accompanied by some volume change of the sand deposits which is manifested on the ground surface as settlements. However, because of non homogenous conditions in the soil deposits, the settlements seldom occur uniformly even in small localized areas and differential settlements become major cause of the damage

to lifelines or other facilities. Although the exact assessment of differential settlements seems beyond the reach of the current-state-of-the-art, they are probably related to the total or average amount of settlements which occur in limited areas being considered.

2.4 Effects of porous media thickness and its hydraulic gradient history on the formation of sand boils: Experimental investigation [4]

In [4] Sand boils occur where groundwater discharges to the land surface under sufficient hydraulic gradient to cause internal erosion and the upward transport of particles. A proper understanding of sand boil processes is essential in evaluating a wide range of geomechanical and sediment transport situations under which groundwater seepage occurs, such as the effects of groundwater discharge on beach stability. Although various empirical methods have been developed to estimate the critical hydraulic gradient (i_{cr}) leading to sand liquefaction, a prerequisite for sand boil occurrence, the effect of sand layer thickness and the implications of driving head fluctuations on the formation and reformation of sand boils have not been explored previously.

This paper uses laboratory experiments to study sand boil formation and reformation for various sand thicknesses and hydraulic gradients to fill this knowledge gap. Sand layer thicknesses of 90 mm, 180 mm and 360 mm were adopted in evaluating sand boil reactivation, which was created by imposing hydraulic head fluctuations. While the first experiment (i.e., 90 mm sand layer) yielded a value for i_{cr} smaller (by 5%) than Terzaghi's (1922) value, the same theory underestimated i_{cr} by 12% and 4% for 180 mm and 360 mm sand layers, respectively. Moreover, i_{cr} needed for the reformation of sand boils decreased by 22%, 22% and 26% (relative to i_{cr} applicable to the initial sand boil) for the 90 mm, 180 mm and 360 mm sand layer thicknesses, respectively. We conclude that the formation of sand boils requires consideration of sand depth and the history of sand

boil formation, particularly in relation to sand boils that form (and potentially reform) under oscillating pressures (e.g., tidal beaches).

Sand column experiments were conducted under controlled laboratory conditions to recreate sand boils in internally stable sand of varying layer thickness. The experimental results showed that thicker sand layers required higher hydraulic gradients (i) for sand boil formation than those predicted by Terzaghi's (1922) equation for the critical hydraulic gradient for fluidization (icr). This reveals the importance of sand layer thickness in sand boil formation, which is neglected in existing formulae for predicting the conditions under which a sand boil may arise. Experimental observations prior to the sand boil emergence indicate that heave is more likely to be a pre-cursor to sand boil formation in experiments with a thicker sand layer (360 mm), with no heave observed for the thinnest sand layer (90 mm).

2.5 Observations on SandBoils from Simple Model Tests [5]

In [5], Earthquake-induced soil liquefaction has been recognized as a hazard for over forty years (e.g. Seed and Lee, 1966; Ishihara, 1985) and static liquefaction for longer (e.g. Koppejan et al., 1948). Loose saturated sands attempting to settle during shaking are unable to dissipate fluid from the contracting void spaces quickly enough, and individual soil grains lose contact with each other and go into suspension (as observed by, e.g., Sasaki et al., 2001). The soil consequently loses strength and dramatic failures occur via a number of mechanisms such as the soil's loss of strength (Fig. 2.2 a) or sand boils (Fig. 2.2 b), bursts of liquefied sand spilling over the ground surface

Sand boils were observed in all tests. Fig. 2.2 shows how these appeared during testing, as thin vertical channels through the silt layer that may or may not contain sand. These often appeared to widen as they approached the surface. Ejection was often sufficiently violent to disturb the

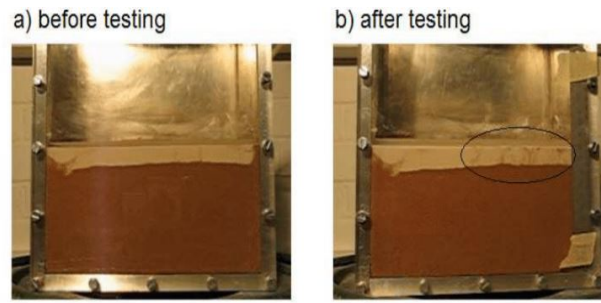


Figure 2.2: Illustration of sand boils using before and after images of test DM1

surface of the water (which was always above the soil surface) indicating great speed and pressure release despite the small-scale nature of the test. These always occurred after the two seconds applied shaking had finished. Further experiments (not presented - Ritchie, 2006; Moran, 2007) in which the same and similar models were subjected to increasingly longer duration shaking showed that an exceptionally long duration of shaking was required to cause boiling during shaking. This may be because the required redistribution of fluid required a much greater length of time than the duration of shaking. This correlates with many eyewitness accounts of earthquakes.

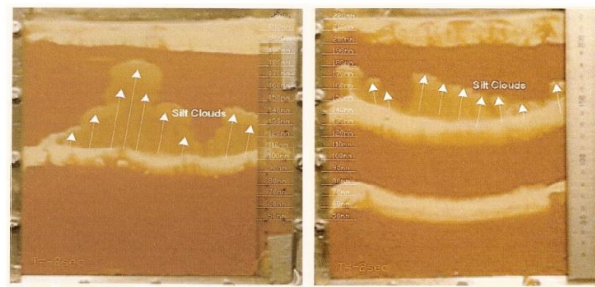


Figure 2.3: Illustration of “silt clouds” from tests a) DM2; b) DM3.

Silt clouds in Fig 2.3 appeared to be due to the top surface of the silt being entrained in suspension by the pore fluid. On the one occasion that a silt cloud hit the base of the surface layer, a sand boil began at the place and moment of impact. This suggests that silt clouds, while not necessarily of engineering importance, have information to give that can be utilized in the future.

2.6 Object Detection [6]

In [6] Image recognition allows machines to identify objects, people, entities, and other variables in images. It is a sub-category of computer vision technology that deals with recognizing patterns and regularities in the image data, and later classifying them into categories by interpreting image pixel patterns. Image recognition includes different methods of gathering, processing, and analyzing data from the real world. As the data is high-dimensional, it creates numerical and symbolic information in the form of decisions. The process of classification and localization of an object is called object detection. Once the object's location is found, a bounding box with the corresponding accuracy is put around it. Depending on the complexity of the object, techniques like bounding box annotation, semantic segmentation, and key point annotation are used for detection. Steps include Data Collection, Pre-processing of the image data, Model architecture and training process, Traditional machine learning algorithms for image recognition.

Some of popular deep learning models for image recognition includes:

YOLO (You Only Look Once) : This object detection algorithm uses a confidence score and annotates multiple objects via bounding boxes within each grid box. YOLO, as the name suggests, processes a frame only once using a fixed grid size and then determines whether a grid box contains an image or not.

Single-shot detector (SSD) : Single-shot detectors divide the image into a default number of bounding boxes in the form of a grid over different aspect ratios. The feature map that is obtained from the hidden layers of neural networks applied on the image is combined at the different aspect ratios to naturally handle objects of varying sizes.

Other machine learning algorithms include Fast RCNN (Faster Region-Based CNN) which is a region-based feature extraction model—one of the best performing models in the family of CNN.

2.7 Machine Learning Methods [7]

In [7] various machine learning methods are described required for sand-boil detection:

Viola-Jones' Method:

In [7] The Viola-Jones' object detector uses AdaBoost algorithm as the learning algorithm. Using OpenCV, Haar features are calculated and gathered. These are then passed through a cascading architecture. The detector starts with a sub-window in each image. This subwindow slides over the image in incremental steps. Windows are checked at every position and scale. They are then passed off to a layer in the cascade filter for processing. The layers vote the window whether it contains the required object or not. If it does not contain the object searched for, then the window is rejected.

You Only Look Once (YOLO) Detection:

In [7] You Only Look Once, or YOLO is a popular object detection algorithm based on the concept of Region Proposal Networks. It is a deep learning framework based on TensorFlow and consumes a lot of computational power. The object detection task at hand is to determine the location of the image where the sand boil is present. Previous methods consisted of pipelines that had multiple steps to perform this very same task. It predicts the presence of sand boils from individual image pixels, the bounding box coordinates and class probabilities of each region or cell. Any input image given to this dense neural network will output a vector of bounding boxes and class predictions.

Single Shot Multibox Detection:

In [7] The single shot multibox detector is an improvement on the YOLO object detector in the sense that it does not have to traverse through the image twice like YOLO does. It can map out the object in a single shot. It is a very fast approach and extremely accurate. SSD is also a deep

learning method for object detection.

The major difference between the YOLO and SSD architectures is that the SSD grids range from very fine to very coarse. This gets more accurate predictions of objects that are of a wider variety of scales. Another major difference can be seen in Figure 2.4.

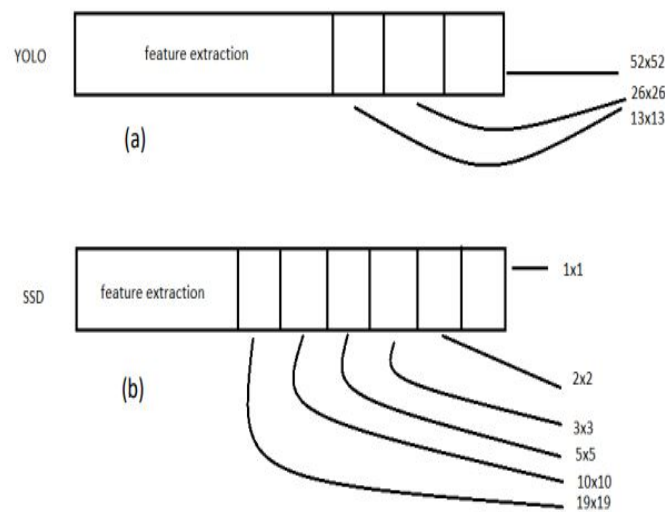


Figure 2.4: Difference between SSD and YOLO methods

Support Vector Machine (SVM):

In [7] A Support Vector Machine (SVM) is a discriminative classifier which is defined by a separating hyperplane. In this algorithm, each data point is plotted as a point in an n -dimensional space (where n is the number of features) with the values of each feature being the value of a coordinate in the space. In order to separate these two classes of data points, many different hyperplanes exist that can be chosen. The objective of SVM is to find the plane that has the maximum margin between the data points of each class. Maximizing the margin distance provides some reinforcement so that the future data points can be classified with more confidence.

Logistic Regression (LogReg):

Logistic Regression [7] is a technique for analyzing data where there are one or more independent variables that determine the dependent variable (outcome). Logistic regression measures the relationship between the dependent variable, in our case: whether the image contains a sand boil or not, and one or more independent variables by generating an estimation probability using logistic regression. It utilizes the sigmoid function to predict the output. In most cases, the dependent variable is a dichotomous variable (in which there are only two possible outcomes).

Extra Tree (ET):

The extremely randomized tree or ET [7] is one of the ensemble methods, which constructs randomized decision trees from the original learning sample and uses above average decision to improve the predictive accuracy and control over-fitting. The Extra-Tree method (standing for extremely randomized trees) was proposed with the main objective of further randomizing tree building in the context of numerical input features, where the choice of the optimal cut-point is responsible for a large proportion of the variance of the induced tree. The method drops the idea of using bootstrap copies of the learning sample and it selects a cut-point at random. From a statistical point of view, dropping the bootstrapping idea leads to an advantage in terms of bias, whereas the cut-point randomization has often an excellent variance reduction effect. This method has yielded state-of-the-art results in several high-dimensional complex problems.

Random Decision Forest (RDF):

Random Decision Forests [7] are an ensemble learning method for classification, regression, and other tasks. RDF operated by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees. Random decision forests correct for decision trees habit of overfitting to their training set. Random Forest adds additional

randomness to the model while growing the trees. Instead of searching for the most important feature while splitting a node, it searches for the best feature among a random subset of features. This results in a wide diversity that generally results in a better model. The RDF operates by constructing a multitude of decision trees on various sub-samples of the dataset and results in the mean prediction of the decision trees to improve the prediction accuracy and control over-fitting.

Bagging (BAG):

Bagging [7] is a “bootstrap” ensemble method that creates individuals for its ensemble by training each classifier on a random redistribution of the training set. Each classifier’s training set is generated by randomly drawing, with replacement, N examples - where N is the size of the original training set; many of the original examples may be repeated in the resulting training set (a) (b)(c) while others may be left out. Each individual classifier in the ensemble is generated with a different random sampling of the training set and subsequently aggregates their individual predictions to yield a final prediction. It is useful for reducing variance in the prediction.

eXtreme Gradient Boosting (XGB):

The implementation of XGB [7] offers several advanced features for model tuning, computing environments, and algorithm enhancement. It can perform the three main forms of gradient boosting (Gradient Boosting (GB), Stochastic GB and Regularized GB) and it is robust enough to support fine-tuning and addition of regularization parameters. However, XGB uses a more regularized model formalization to control over-fitting, which results in better performance. In addition to better performance, XGB is designed to provide higher computational speed.

2.8 Viola-Jones' Object Detector [8]

In [8] In this article, we decipher the Viola-Jones algorithm, the first ever real-time face detection system. There are three ingredients working in concert to enable a fast and accurate detection: the integral image for feature computation, Adaboost for feature selection and an attentional cascade for efficient computational resource allocation. Here we propose a complete algorithmic description, a learning code and a learned face detector that can be applied to any color image. Since the Viola-Jones algorithm typically gives multiple detections, a post-processing step is also proposed to reduce detection redundancy using a robustness argument. A waterfall model of cascade layers filters out as many negatives as possible in the shortest time possible. If any of the windows contain a positive image, then they are retained and passed on to the next classifier. This means that the windows discarded in the first few stages are most likely to be negative. Most true negatives are collected in the initial stages of the cascade. In the subsequent steps, the remaining positives are closely examined and determined to be true positives.

Haar cascades are known to be one of the fastest and accurate methods for face detection, even if it is relatively an old algorithm (2001). A cascade's detection rate is equal to the product of every layer's detection rates. As in, for a cascade with 4 layers with each layer's detection rate at 0.99, the final rate would be 0.99 raised to the power of 4, which is 0.96. The detection rate decreases in subsequent layers because every layer is unable to achieve a 100% detection rate. This causes the cascade to lose some positive images. Hence, the number of false negative detection increases for a very high number of cascade stages. Viola Jones' object detection algorithm achieves real-time detection of objects. This makes it an extremely popular algorithm to run on mobile devices and cameras. Figure 2.5 describes the Viola-Jone's algorithm.

Any subimage that is voted as not being a sand boil is rejected and the image voted as a sand boil is carried over to the next stage which performs the same function once more. In this iterative process, most of the nega-

tive image samples are weeded out successfully. The samples that are left are assumed to be positive sample.

To implement the algorithm, Open-source Computer Vision (OpenCV) as a Python package is used. OpenCV contains functions for use in both C++ and Python for image manipulation. In this research, we use it to construct a dataset, run the cascade training and test a set of images for recognition.

2.9 You Only Look Once (YOLO) Object Detector [9]

In [9] YOLO reframes the object detection pipeline of older papers such as R-CNN, Faster-RCNN as a single regression problem. Within YOLO, the input image is divided into a grid of $S \times S$ cells. For every object that is within the image, one grid cell is responsible for detecting it. The cell responsible is the one where the center of the object falls into. Each cell predicts B bounding boxes and C class probabilities.

The bounding box prediction has 5 components – (x , y , w , h , confidence score). Here, (x , y) coordinates represent the center of the box in relation to the grid cell location. The cell division and prediction responsibilities of each grid cell. The image bounded inside the red box is the sand boil area that needs to be detected. Since the center of that square lies within the grid cell in the center, it is now responsible for making the prediction of the sand boil. Each of these grid cells makes a total of $S \times S \times B \times 5$ predictions, as described earlier. It is also important to predict the class probabilities for each cell. These conditional probabilities $\Pr(\text{class}(i) \text{ — Object})$ are necessary to create a region proposal network.

YOLO is refreshingly simple. A single convolutional network simultaneously predicts multiple bounding boxes and class probabilities for those boxes. YOLO trains on full images and directly optimizes detection performance. This unified model has several benefits over traditional methods of object detection. First, YOLO is extremely fast. Since we frame detec-

tion as a regression problem we don't need a complex pipeline. We simply run our neural network on a new image at test time to predict detections. Our base network runs at 45 frames per second with no batch processing on a Titan X GPU and a fast version runs at more than 150 fps. This means we can process streaming video in real-time with less than 25 milliseconds of latency. Furthermore, YOLO achieves more than twice the mean average precision of other real-time systems.

Second, YOLO reasons globally about the image when making predictions. Unlike sliding window and region proposal-based techniques, YOLO sees the entire image during training and test time so it implicitly encodes contextual information about classes as well as their appearance. Fast R-CNN, a top detection method mistakes background patches in an image for objects because it can't see the larger context. YOLO makes less than half the number of background errors compared to Fast R-CNN. Third, YOLO learns generalizable representations of objects. When trained on natural images and tested on artwork, YOLO outperforms top detection methods like DPM and R-CNN by a wide margin. Since YOLO is highly generalizable it is less likely to break down when applied to new domains or unexpected inputs.

2.10 Single Shot Multibox Detector [10]

In [10] Single shot multibox detector can localize the object in a single forward pass of the network. The single shot multibox detector's architecture builds on the VGG-16 architecture but discards the fully connected layers. VGG-16 performs very well on high-quality image classification. However, in this research, we use the MobileNetV2 architecture in the place of VGG-16. MobileNetV2 is one of the most recent mobile architectures (published by Google in March 2019). It favors speed over accuracy but still manages to output excellent detections. Features are extracted at multiple scales by the MobileNet network and forward passed to the rest of the network. In MultiBox, researchers created priors (popularly known as anchors in Faster R-CNN terms), which are values of fixed size bounding boxes that

are pre-computed and closely match the original ground truth boxes. Priors are chosen in a way such that the ratio of original ground truth boxes to the prediction is greater than 0.5. This is also called IoU (intersection over union). In SSD, priors are fixed manually. For each prediction of a bounding box, a set of C class predictions are computed for every possible class in the dataset.

Our approach, named SSD, discretizes the output space of bounding boxes into a set of default boxes over different aspect ratios and scales per feature map location. At prediction time, the network generates scores for the presence of each object category in each default box and produces adjustments to the box to better match the object shape. Additionally, the network combines predictions from multiple feature maps with different resolutions to naturally handle objects of various sizes. SSD is simple relative to methods that require object proposals because it completely eliminates proposal generation and subsequent pixel or feature resampling stages and encapsulates all computation in a single network. This makes SSD easy to train and straightforward to integrate into systems that require a detection component. Experimental results on the PASCAL VOC, COCO, and ILSVRC datasets confirm that SSD has competitive accuracy to methods that utilize an additional object proposal step and is much faster, while providing a unified framework for both training and inference.

For 300x300 input, SSD achieves 74.3% mAP on VOC2007 test at 59 FPS on a Nvidia Titan X and for 500x500 input, SSD achieves 76.9% mAP, outperforming a comparable state of the art Faster R-CNN model. Compared to other single stage methods, SSD has much better accuracy even with a smaller input image size.

2.11 Stacking [11]

In [11] A total of 8 different methods were used, including Support Vector Machine (SVM) which is a discriminative classifier which is defined by a separating hyperplane, Logistic Regression which is a technique for

analyzing data where there are one or more independent variables that determine the dependent variable (outcome), Extremely randomized Tree or ET which constructs randomized decision trees from the original learning sample and uses above-average decision to improve the predictive accuracy and control over-fitting, Random Decision Forests which is an ensemble learning method for classification, regression, and other tasks, K nearest neighbors which stores all available cases and classifies new cases based on a similarity measure, Bagging which creates individuals for its ensemble by training each classifier on a random redistribution of the training set, Gradient Boosting Classifier (GBC) which builds an additive model in a forward stage-wise fashion, and eXtreme Gradient Boosting (XGB) that has more advanced features for model tuning.

The comparison of different object detection methods was done in Table 2.2 and they determined the best ones to use for the detection of sand boils. They have also developed a Stacking-based machine learning predictor which focuses on using the best methods to increase the detection accuracy of the machine learning model.

Stacking based machine learning approaches [11] have been very successful when applied to some interesting problems. This idea is utilized here to try and develop a better performing sand boil predictor. Stacking is an ensemble approach that obtains information from multiple models and aggregates them to form a new and generally improved model. 18 different stacking models were examined as described in Table 2.1. These models are built and optimized using Scikit-Learn . To select the algorithms to be used as base classifiers, we evaluate all these combinations.

2.12 Summary

The literature survey on sandboil detection through image processing has witnessed the application of well-established object detection frameworks, namely Viola-Jones, YOLO (You Only Look Once), and SSD (Single Shot Multibox Detector). These frameworks, originally developed for diverse

Method	Sensitivity	Specificity	Accuracy	Precision	F1 Score	MCC
SVM	97.49	92.05	94.77	92.46	0.949	0.8967
KNN	61.82	81.9	71.86	77.35	0.6872	0.4463
GBC	97.28	88.49	92.88	89.42	0.9318	0.8610
XGBC	89.43	89.33	89.38	89.34	0.8938	0.7876
RDF	92.46	89.74	91.11	90.02	0.9122	0.8224
ET	95.5	90.48	92.99	90.93	0.9316	0.8609
LOGREG	81.27	81.79	81.53	81.7	0.8148	0.6307
BAGGING	92.05	87.34	89.69	87.91	0.8993	0.7958

Table 2.1: Accuracies of Different Methods

object recognition tasks, have been customized to address the specific challenges associated with identifying sandboils in remote sensing imagery. These methods are integrated into a comprehensive workflow that involves image preprocessing, feature extraction, object detection, and classification. Image preprocessing aims to enhance relevant features and normalize the data, while feature extraction captures the unique characteristics of sandboils. The subsequent object detection phase employs the Viola-Jones, YOLO, or SSD frameworks to identify sandboils as objects of interest within the images. Post-detection, classification techniques may be applied to differentiate between sandboils and other objects or background elements.

Validation and accuracy assessment are crucial components of these methodologies. Researchers evaluate the performance of the adapted frameworks using standard metrics such as precision and recall. Ground truth data and field surveys contribute to the validation process, ensuring the reliability and effectiveness of the image processing techniques for sandboil detection.

In conclusion, the literature survey underscores the versatility of object detection frameworks like Viola-Jones, YOLO, and SSD in addressing the challenges of sandboil detection through image processing. These frame-

Related Literature	Techniques	Advantages	Limitations
[7]	SVM Method	Works relatively well when there is a clear margin of separation between classes.	Long training time for large datasets. Difficult to understand and interpret the final model.
[7]	Bagging Method	Best for data that has high variance as it can reduce overfitting and increase model's stability.	It introduces a loss of interpretability of a model.
[8]	Viola Jones' Method	Improves tracking speed by removing the need to redetect objects in each frame.	It is not effective while detecting tilted or turned objects.
[9]	YOLO Method	Its fast inference speed allows it to process images in real time.	Difficult to detect small objects from the image and which are very close to each other because of the grid.
[10]	SSD Method	Maintains accuracy while improving speed by eliminating the need for resampling the pixels or features.	Difficulty in detecting tiny objects, heavily occluded objects or objects with extreme aspect ratios.
[11]	Stacking	Better prediction accuracy compared to bagging method.	Since it combines bagged models it needs more time and computational power.

Table 2.2: Comparison of different methods

works offer efficient and real-time solutions for monitoring and managing flood-affected areas, contributing to the advancement of technology-driven strategies in disaster response and mitigation.

2.13 Proposed System

2.13.1 Why the chosen problem / project is important?

The identification of sandboils stands as a pivotal component in assessing geological and geotechnical risks, especially in areas susceptible to flooding, seismic events, or elevated groundwater levels. Sandboils, a geological occurrence, manifest when pressurized water pushes up, bringing along fine sand and silt. Such events are often linked with factors like floods, earthquakes, or disturbances that heighten water pressure below ground

2.13.2 What is novel in proposed project work?

The novelty of the proposed project lies in its endeavor to introduce a more efficient and proactive approach to sand boil detection. While existing manual methods are often slow, costly, and lack early warning capabilities, this project seeks to revolutionize the process through an automated system and predictive model leveraging image processing techniques. By doing so, it not only aims to enhance the speed of detection but also to foster timely interventions, ultimately bolstering infrastructure resilience and safeguarding public well-being.

2.13.3 How would it advance the state-of-the-art?

The proposed project would advance the state of the art by introducing a cutting-edge automated system and predictive model for sand boil detection. By harnessing image processing techniques, it would elevate the precision, speed, and early warning capabilities of detection methods. This advancement not only streamlines the process but also introduces a proactive dimension to geotechnical hazard management, setting a new benchmark for efficiency and effectiveness in the field.

2.13.4 How does your tool/technique/approach differ from each of the existing works that you have surveyed?

The proposed technique differs from existing methods by integrating advanced image processing techniques to automate sand boil detection. Unlike traditional methods that may be time-consuming or lack early warning capabilities, this approach seeks to provide a more efficient, timely, and proactive solution to address the identified challenges.

Chapter 3

Methodology

3.1 Algorithms

3.1.1 Viola Jones

Viola-Jones algorithm is a machine-learning technique for object detection proposed in 2001 by Paul Viola and Michael Jones in their paper “Rapid object detection using a boosted cascade of simple features”. The algorithm was primarily conceived for face detection but is also an efficient solution for resource-constrained devices.

Given a grayscale image, the algorithm analyzes many windows of different sizes and positions and tries to detect the target object by looking for specific image features in each window. The Viola-Jones algorithm is based on four main ideas: Haar-like features Integral images for accelerating the feature computation AdaBoost learning for feature selection Cascade of classifiers for fast rejection of windows without faces.

In [7] Open Source Computer Vision (OpenCV) was used to run the viola jones algorithm. Firstly, the positive and negative samples of sand boils were sorted into different folders. Then, we extracted the information about coordinates (x, y, w, h) where x, y are the top corner and w, h are the width and height of the object inside the image. Then, the cascade is trained using 25 stages, 4500 positive samples, and 3000 negative samples. This results in an xml file that can be used to make the predictions on a test dataset. The test dataset contains a total of 8300 images out of which 2000 are negative samples, and the rest are positive samples of sand

boils. The cascade training is stopped at stage 25, and the intermediate stages, 10, 15, 20 and 25 are tested for performance. The best performance is achieved by cascade stage 15. This is evidenced in the results section. After this, the accuracy is measured, and bounding boxes are drawn based on the predicted outputs.

Algorithm 3: Viola-Jones Cascading Classifier

```

1  input: target false positive rate  $fp$ 
2  input: false positive rate per layer  $fp_i$ 
3  input: detect rate per layer  $dt_i$ 
4  input : set of positive and negative images  $j$ 
5  variable  $i = 0$  starting detection rate  $d = 1.0$ ; starting false positive rate  $f = 1.0$ 
6  while  $f < fp$ :
7       $i++$ 
8       $n = 0$ 
9      while  $fp_m < f * fp_{i-1}$ 
10          $n++$ 
11         collection of weak classifiers  $m = \text{Ada-Boost}(n, j)$ 
12         evaluate collection on test set of images
13         while  $dt_m < d * dt_{i-1}$ 
14             decrease threshold  $k$  for collection and re-evaluate
15     set  $m$  as a layer  $i$  for cascaded classifier
16     delete any negatives in  $j$  correctly classified

```

Figure 3.1: Viola-Jones' Algorithm

3.1.2 You Only Look Once (YOLO)

You Only Look Once (YOLO) is one of the most popular model architectures and object detection algorithms. It uses one of the best neural network architectures to produce high accuracy and overall processing speed, which is the main reason for its popularity.// YOLO algorithm aims to predict a class of an object and the bounding box that defines the object location on the input image. It recognizes each bounding box using four numbers:

Center of the bounding box (b_x, b_y)

Width of the box (b_w)

Height of the box (b_h)

In addition to that, YOLO predicts the corresponding number c for the predicted class as well as the probability of the prediction (P_c)

In [7], For YOLO, the configuration of the basic net was left unchanged

1. **Data Preparation:**
 - Gather and preprocess the dataset, including resizing images to a standard size and organizing them into appropriate directories (e.g., train, validation, test).
2. **Model Initialization:**
 - Initialize the YOLOv8 model architecture. YOLOv8 typically consists of a backbone network (e.g., Darknet), followed by classification layers.
3. **Training:**
 - Iterate over the training dataset in batches.
 - Forward pass: Input batch images to the model and obtain predictions.
 - Calculate the loss between predicted probabilities and ground truth labels.
 - Backward pass: Update the model's weights using gradient descent to minimize the loss.
4. **Validation:**
 - Periodically evaluate the model's performance on a separate validation dataset.
 - Adjust hyperparameters (e.g., learning rate) based on validation performance to improve the model's generalization ability.
5. **Testing:**
 - Evaluate the trained YOLOv8 model on a separate test dataset to assess its performance and generalization to unseen data.
6. **Deployment:**
 - Deploy the trained YOLOv8 model for inference on new images, allowing it to classify objects present in the images.

Figure 3.2: YOLO Algorithm

except for the width, height and the number of classes. A TensorFlow implementation of the darknet, called Darkflow was used to run YOLO. Darknet is an open source neural network framework written in C language. The architecture of the detector consists of 24 convolutional layers followed by 2 fully connected layers. Alternating 1 x 1 convolutional layers reduce the feature space from the earlier layers. These layers are pre-trained on the ImageNet classification dataset. For the problem, they chose to train the network from scratch, without initializing the weights for the network. The average moving loss starts at a very high value and slowly reduces. The network must be trained until the loss falls below 0.01 in order to get the best bounding boxes. The training ran for around 2 weeks on a server, at which point the loss function was extremely low. At this point, the latest checkpoint is used to test the detections on a test

dataset of 112 images. After the detections are made, the accuracy can be measured by comparing the ground truth files with the predicted outputs.

3.1.3 Support Vector Machine (SVM)

Support Vector Machine (SVM) is a powerful and versatile machine learning algorithm used for both classification and regression tasks.[7]. Its main objective is to find the optimal hyperplane that separates different classes in the feature space while maximizing the margin between classes. SVM works by mapping input data into a high-dimensional space and then finding the hyperplane that best separates the classes.

Here's how SVM works:

Data Preparation: SVM works with labeled data, meaning you need a dataset where each data point is associated with a class label. Each data point should also have features that describe it.

Feature Space: SVM maps each data point into a high-dimensional feature space. In this feature space, each feature corresponds to a dimension, and the data points are represented as vectors.

Finding the Optimal Hyperplane: The goal of SVM is to find the hyperplane that best separates the data points into different classes while maximizing the margin, which is the distance between the hyperplane and the closest data points (support vectors). This hyperplane is the one that maximizes the margin and minimizes the classification error.

Margin Optimization: SVM optimizes the margin by solving a constrained optimization problem. The optimization objective is to maximize the margin while ensuring that all data points are correctly classified or fall within a certain margin of the hyperplane.

Kernel Trick: In cases where the data is not linearly separable in the original feature space, SVM can use a kernel trick to map the data into a higher-dimensional space where it becomes linearly separable. Common kernel functions include linear, polynomial, radial basis function (RBF), and sigmoid kernels.

Classification: Once the optimal hyperplane is found, SVM can classify

new data points by determining which side of the hyperplane they fall on.

1. Define your training dataset (images and corresponding labels)
2. Preprocess the images (e.g., resize, normalize)
3. Define the SVM model:

SVM_Model:

 - Initialize weights w and bias b
 - Define hyperparameter C (regularization parameter)
 - Define kernel function $K(x, y)$ (e.g., linear kernel, Gaussian kernel)
4. Train the SVM model:

Train_SVM_Model(images, labels):

For each image-label pair (x_i, y_i) in training dataset:

For each training iteration:

 - Compute the predicted class $\hat{y} = \text{sign}(K(x_i, w) + b)$
 - Update weights and bias based on hinge loss:
 - $w = w - \alpha * (\partial L / \partial w + C * \partial R / \partial w)$
 - $b = b - \alpha * (\partial L / \partial b)$

where L is the hinge loss, R is the regularization term, and α is the learning rate
5. Classify new images:

Classify_Image(image):

 - Preprocess the image
 - Compute the predicted class $\hat{y} = \text{sign}(K(\text{image}, w) + b)$
 - Return the predicted class label
6. Evaluate the model performance (e.g., accuracy, precision, recall) on a separate test dataset

Figure 3.3: SVM Algorithm

Overall this algorithm is particularly effective in scenarios with complex decision boundaries and works well even in high-dimensional spaces. SVM has various kernel functions (e.g., linear, polynomial, radial basis function) that allow it to handle nonlinear relationships between features. SVM is widely used in various fields, including image classification, text classification, and bioinformatics, due to its robustness and ability to generalize well to unseen data.

3.1.4 Convolution Neural Network (CNN)

The Convolutional Neural Network (CNN) is a class of deep neural networks primarily used for analyzing visual imagery. Inspired by the organization of the animal visual cortex, CNNs consist of multiple layers of interconnected neurons that perform hierarchical feature extraction. The

key components of a CNN include convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply filters to input images to extract features like edges and textures, while pooling layers downsample feature maps to reduce computational complexity. Finally, fully connected layers combine the extracted features to make predictions. CNNs have revolutionized various computer vision tasks, including image classification, object detection, and image segmentation, due to their ability to automatically learn discriminative features from raw pixel data.

1. Define your training dataset (images and corresponding labels)
2. Preprocess the images (e.g., resize, normalize)
3. Define the CNN architecture:

 CNN_Model:
 - Initialize the CNN architecture (e.g., number of convolutional layers, filter sizes, pooling layers, fully connected layers)
 - Define hyperparameters (e.g., learning rate, batch size, number of epochs)
4. Train the CNN model:

 Train_CNN_Model(images, labels):
 For each epoch:
 For each batch of images and labels:
 - Forward pass:
 - Apply convolutional layers (with activation functions)
 - Apply pooling layers (e.g., max pooling)
 - Flatten the output
 - Apply fully connected layers (with activation functions)
 - Apply softmax layer to get probabilities
 - Compute loss using cross-entropy loss function
 - Backward pass:
 - Compute gradients using backpropagation
 - Update weights and biases using gradient descent or its variants (e.g., Adam optimizer)
5. Classify new images:

 Classify_Image(image):
 - Preprocess the image
 - Forward pass through the trained CNN model
 - Return the class label with the highest probability
6. Evaluate the model performance (e.g., accuracy, precision, recall) on a separate test dataset.

Figure 3.4: CNN Algorithm

CNNs are trained using a large dataset of labeled images, where the network learns to recognize patterns and features that are associated with specific objects or classes. Proven to be highly effective in image-related tasks, achieving state-of-the-art performance in various computer vision applications. Their ability to automatically learn hierarchical representa-

tions of features makes them well-suited for tasks where the spatial relationships and patterns in the data are crucial for accurate predictions

CNNs are trained using a supervised learning approach. This means that the CNN is given a set of labeled training images. The CNN then learns to map the input images to their correct labels. The training process for a CNN involves the following steps:

Data Preparation: The training images are preprocessed to ensure that they are all in the same format and size.

Loss Function: A loss function is used to measure how well the CNN is performing on the training data. The loss function is typically calculated by taking the difference between the predicted labels and the actual labels of the training images.

Optimizer: An optimizer is used to update the weights of the CNN in order to minimize the loss function.

Backpropagation: Backpropagation is a technique used to calculate the gradients of the loss function with respect to the weights of the CNN. The gradients are then used to update the weights of the CNN using the optimizer.

3.2 Design

Detecting sand boils, which are indicative of potential levee breaches or weaknesses, can be crucial for infrastructure protection and disaster mitigation. Image processing techniques can be effectively utilized for this purpose. Here's the design outline for sand boil detection using image processing:

- 1. Image Acquisition:** Obtain images of the levee area using drones equipped with high-resolution cameras or satellite imagery. Alternatively, deploy fixed cameras strategically along the levee for continuous monitoring.
- 2. Pre-processing:** Correct for lens distortion and adjust image brightness and contrast. Convert images to grayscale to simplify processing.

Apply noise reduction techniques to enhance image quality.

- 3. Segmentation:** Utilize thresholding techniques to separate potential sand boil regions from the background. Experiment with different thresholding methods (e.g., global thresholding, adaptive thresholding) to handle variations in lighting and terrain.
- 4. Feature Extraction:** Extract features such as area, perimeter, circularity, and texture from segmented regions. Use morphological operations (e.g., erosion, dilation) to refine the regions and remove noise.
- 5. Classification:** Train a machine learning model using labeled data to classify segmented regions as either sand boils or background. Use extracted features as input to the classifier.
- 6. Post-processing:** Apply morphological operations to further refine the detected regions and remove false positives. Perform spatial and temporal analysis to validate detection and reduce false alarms. Implement logic to merge overlapping detection and remove isolated outliers.
- 7. Visualization and Reporting:** Visualize the detected sand boils overlaid on the original images or on a map of the levee area. Generate reports with information on the location, size, and confidence level of detected sand boils. Provide alerts or notifications to relevant stakeholders for timely action.
- 8. Integration:** Integrate the sand boil detection system with existing levee monitoring infrastructure. Develop APIs or interfaces for seamless integration with other systems (e.g., Geographic Information Systems, emergency response platforms). Ensure scalability and compatibility with different hardware configurations.
- 9. Validation and Optimization:** Validate the performance of the detection system using ground truth data collected from field inspections. Continuously optimize the algorithms based on feedback and

new insights gained from monitoring activities. Regularly update the machine learning model with new labeled data to improve accuracy and adapt to changing conditions.

- 10. Deployment and Maintenance:** Deploy the sand boil detection system in operational levee monitoring networks. Establish protocols for routine maintenance, calibration, and software updates. Provide training and documentation for operators and maintenance personnel.

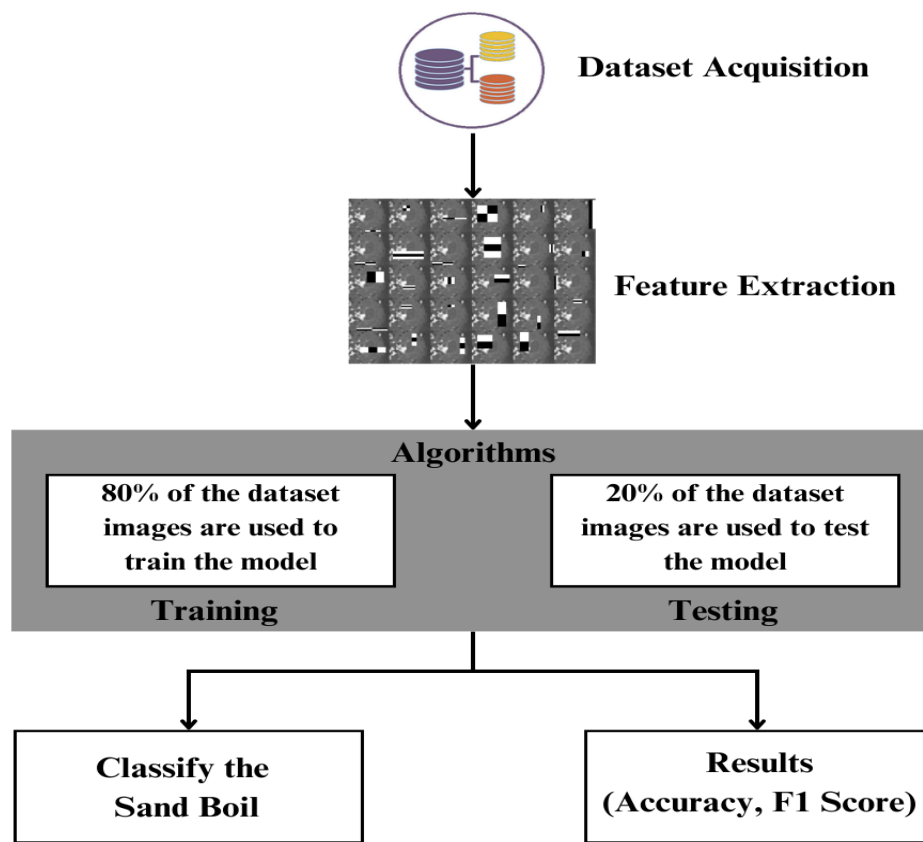


Figure 3.5: Architecture Diagram

This diagram represents a process flow for a machine learning task, specifically for classifying images of sand boils. Here's a step-by-step explanation of the diagram:

Dataset Acquisition: The first step involves collecting a dataset, which is represented by the stack of colored discs. This dataset consists of

images that will be used to train and test a machine learning model.

Feature Extraction: After acquiring the dataset, the next step is to extract features from the images. Feature extraction involves processing the images to identify and quantify relevant characteristics or patterns that the machine learning algorithms can use to learn from the data.

Algorithms: The extracted features are then fed into algorithms. This part of the process is divided into two main phases:

Training: 80% of the dataset images are used to train the model. During training, the algorithm learns to recognize patterns and features that are indicative of sand boils.

Testing: The remaining 20% of the dataset images are used to test the model. Testing evaluates how well the model has learned to classify sand boils by using data it has not seen during the training phase.

Classify the Sand Boil: The primary goal of the model is to classify images as sand boils. This step is likely the application of the trained model to new data to make predictions or classifications.

Results (Accuracy, F1 Score): The final step is to assess the performance of the model using metrics such as accuracy and the F1 score. Accuracy measures the proportion of correct predictions made by the model, while the F1 score is a harmonic mean of precision and recall, providing a balance between the two for situations where uneven class distribution might make accuracy a less reliable metric.

Overall, the diagram outlines a typical supervised machine learning workflow for image classification, from data collection to model evaluation.

Chapter 4

Implementation

4.1 Description of Dataset

In this study, we meticulously curated our dataset by collecting data from a variety of sources. We faced the challenge of not having a centralized repository for levee and sand boil data, prompting us to adopt a creative approach. To ensure a comprehensive representation, we even crafted some images ourselves by overlaying different types of sand boil images onto various backgrounds.

Our dataset comprises images sourced from diverse platforms, including social media and Google Images. Each image was thoughtfully selected to facilitate the nuanced analysis of sand boils using machine learning and image processing techniques. We prioritized the inclusion of both positive and negative samples, recognizing their significance in training machine learning algorithms effectively. Positive samples vividly illustrate instances where sand boils are unmistakable, while negative samples depict scenarios devoid of such features. This balanced assortment is crucial for enabling our algorithms to accurately discern the presence or absence of sand boils.

Moreover, we took deliberate efforts to incorporate challenging cases into our dataset. By doing so, we aimed to enrich our dataset and fortify the models developed in this research.

4.2 Features of the Dataset

- **Appearance:** Sand boils typically appear as small mounds or patches of sand or water erupting from the ground, often in a circular or semi-circular shape.
- **Texture:** The texture of sand boils can vary, ranging from loose, sandy material to muddy or watery consistency.
- **Location:** Sand boils often occur near levees, riverbanks, or areas with high water pressure underground, indicating potential weaknesses in the soil or structure.
- **Association with Water:** Sand boils are often associated with water seepage or saturation in the soil, indicating potential breaches or failures in levees or barriers.
- **Surrounding Environment:** Examination of the surrounding area may reveal signs of soil disturbance, cracking, or erosion, which can be indicative of sand boils.
- **Size:** Sand boils can vary in size, from small, localized eruptions to larger, more extensive formations, depending on factors such as water pressure and soil conditions.
- **Temporal Changes:** Monitoring changes in the size, shape, or intensity of sand boils over time can provide valuable insights into the dynamics of water flow and soil stability.
- **Geological Context:** Understanding the geological characteristics of the area, such as soil composition, water table depth, and historical flood patterns, can help contextualize the presence and behavior of sand boils.



Figure 4.1: Positive Image



Figure 4.2: Positive Image

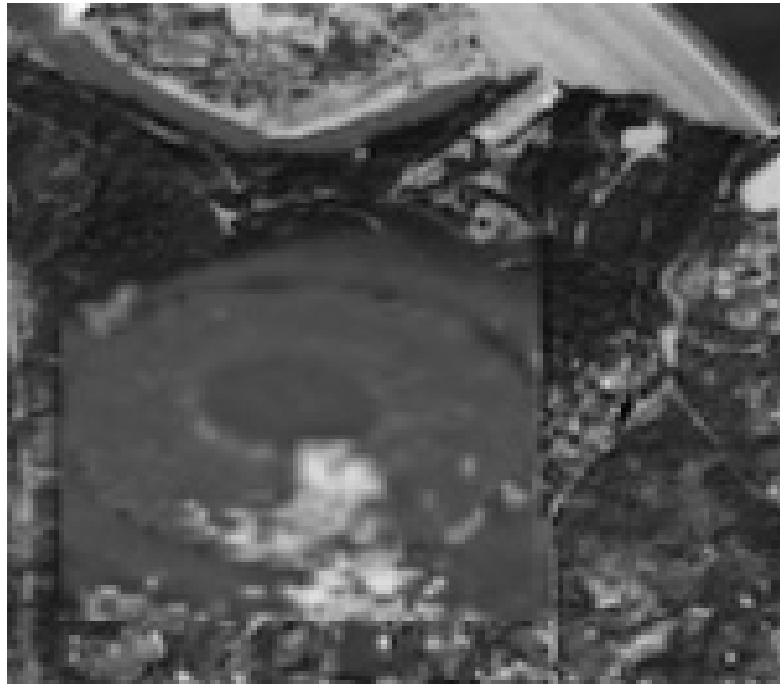


Figure 4.3: Positive Image



Figure 4.4: Positive Image



Figure 4.5: Negative Image



Figure 4.6: Negative Image



Figure 4.7: Negative Image



Figure 4.8: Negative Image

4.3 Experimental Setup

4.3.1 Feature Extraction

In this section, we describe the features from both positive and negative samples that have been extracted and fed into the machine learning methods. For the Viola-Jones algorithm, the features used are called Haar features. These are calculated internally by OpenCV. They can be visualized to see which regions of the image contribute to the classification of it being a positive sample. Further discussion and examples can be found in the results section. For the YOLO algorithm and the SSD method, since the architecture can be considered a part of the convolutional neural network, the features are extracted internally by the convolutional net itself. For the non-deep learning methods, we need to extract some features manually.

4.3.2 Haralick Features

Haralick features or textural features for image classification are very important in identifying the texture characteristics of an object in an image. For aerial photographs or satellite images, these features can isolate the textures and regions of interest of the image being searched for. In conjunction with machine learning methods, these features can be extremely useful in identifying sand boils from satellite imagery. The basis of these features is a gray-level co-occurrence matrix. The matrix is generated by counting the number of times a pixel with a value i is adjacent to a pixel with a value j and then dividing the entire matrix by the total number of such comparisons made. Each entry is therefore considered to be the probability that a pixel with a value i will be found adjacent to a pixel of value j . This feature is rotation invariant and works very well for our case because the structure of a sand boil is predictable conceptually. Haralick features yield a list of 13 features.

4.3.3 Hu Moments

Hu moments, also known as image moments are important features in computer vision and image recognition fields. They are useful for describing object segmentation and other properties. It is a weighted average (or moment) of the image pixel intensities. For a binary image, it is the sum of pixel intensities (all white) divided by the total number of pixels in that image. The moments associated with a shape remain constant even on rotating the image. The Hu moments of K differ from that of S0, S1, ... S4. Whereas, the values for all the images containing S in them happen to be similar. It is also evident that the moment values for the images are rotation invariant. Hence, these are good features for the detection of sand boils. Hu Moments yields a list of 7 features. $H[0]$ is the first Hu moment of K0 (which is a binary image of the letter 'K'). $H[1]$, similarly, is the second Hu moment value for K0 and so on. In cases like sandboils, Hu moments are used because these features are rotation invariant and provide a clear distinction between two types of images, as evidenced in the image. Therefore, the assumption is that for sand boils the values $H[0]$, $H[1]$, ..., $H[6]$ will remain similar to each other and therefore help during testing.

4.3.4 Histograms

Histograms can be generated directly using OpenCV's `calcHist` function in Python or C++. It calculates the histogram of one or more arrays. We use it to generate 2D histograms to understand the distribution of pixel intensities in the image. This proves to be a useful feature for the detection of images based on the assumption that similar images will have similar histograms. The number of bins can be specified in the parameters required. It can range from 32, 64 to 256. The difference between a 32-bin and 256-bin histogram. It yields a list of 32 features.

4.3.5 Histogram of Oriented Gradients (HOG)

Computer vision world has been known for improving the accuracy of detection for images with fixed outlines such as pedestrians. Dalal and Triggs are known for detecting humans using these features. The algorithm counts the occurrences of gradient orientation in localized portions of an image. Similar to edge detection, it describes the local object appearance and shape in an image. The image is divided into small connecting regions and for each pixel in each cell, a histogram of gradient direction is calculated. This results in a final list of appended histograms which is the final list of features.

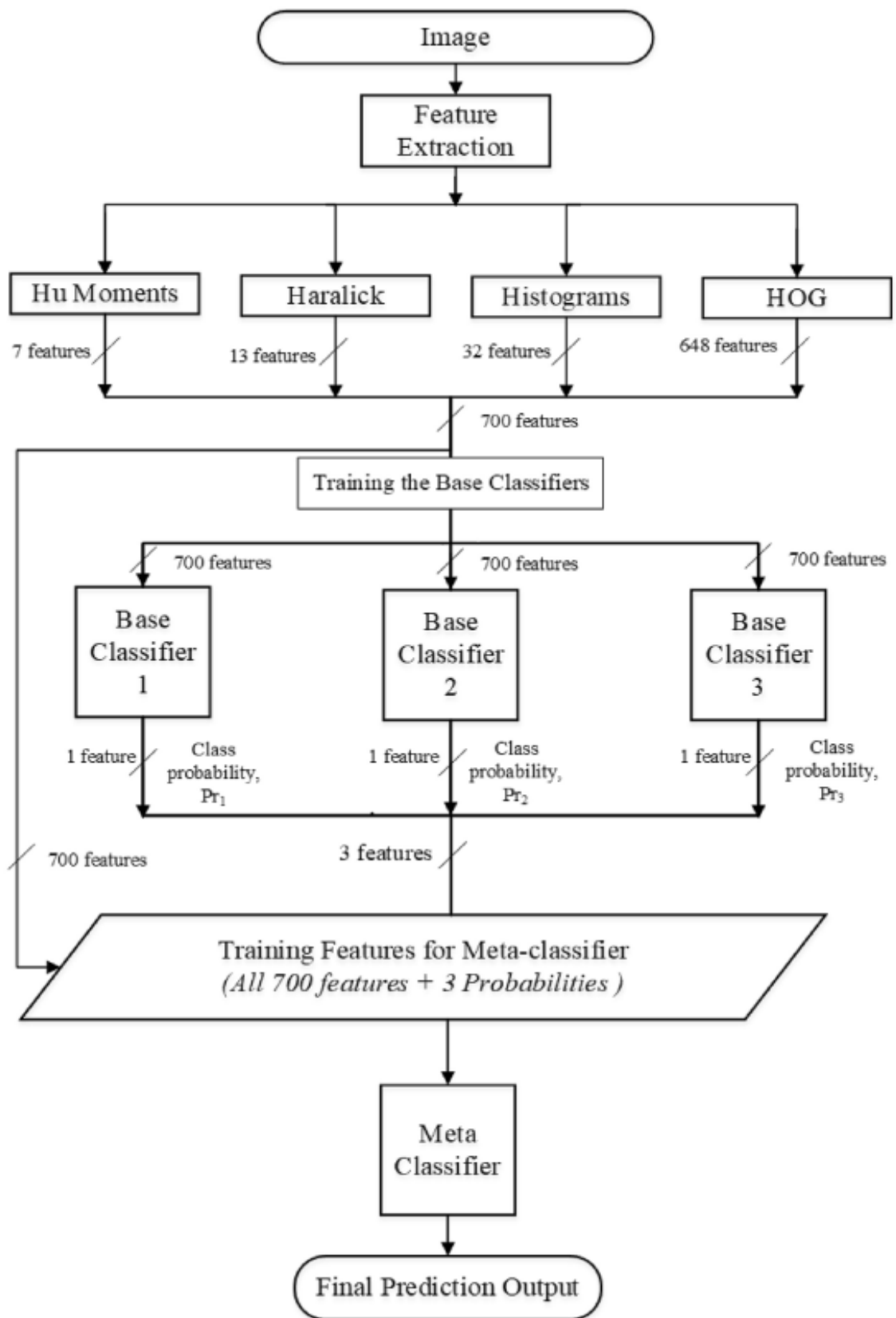


Figure 4.9: Feature Extraction

4.4 Hardware and Software

4.4.1 Software Requirements

Programming Languages: Python for scripting and implementing image processing, machine learning and deep learning algorithms.

Libraries and Frameworks: OpenCV, TensorFlow, Scikit-Learn, NumPy, Keras

Integrated Development Environment (IDE): Jupyter Notebook, Visual Studio, Google Colab

4.4.2 Hardware Requirements

Processor(CPU): i5 or above

RAM(Memory): 18GB or above

Hard disk Space: Multi high-speed SSDs, 20GB+

Display: Predictive Model

Chapter 5

Results and Discussion

5.1 Viola Jones

In the Viola-Jones' object detection algorithm, we achieve an overall accuracy of 86.11%. The precision is 0.8170 and recall is 0.8241. The F1 score is 0.8205. 6305 images were used for training data and 78 images were used for testing data. Overall, the Viola-Jones algorithm performs very well despite being one of the oldest methods for object detection.



Figure 5.1: Viola Jones Algorithm Detecting a Sand Boil



Figure 5.2: Viola Jones Algorithm Detecting a Sand Boil

5.2 YOLO

YOLO yields a set of detections that appear to be correct for a classifier – not as a detector. In the YOLO algorithm, we achieve an overall accuracy of 98.74% which is the highest among the other methods. The precision is 0.9281 and recall is 0.9633. The F1 score is 0.9453. 6303 images were used for training data and 14 images were used for testing data.

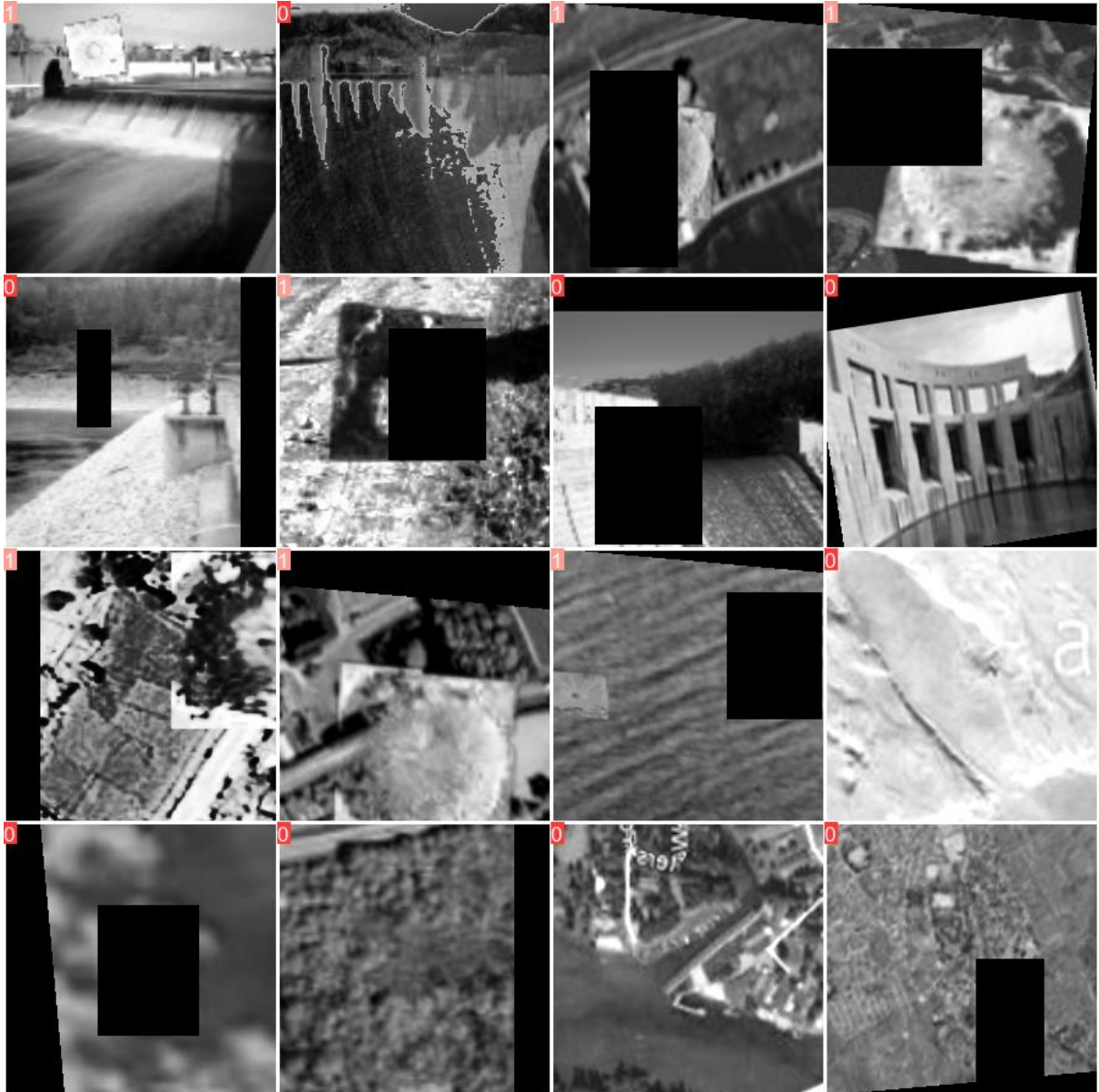


Figure 5.3: YOLO Algorithm Training Images

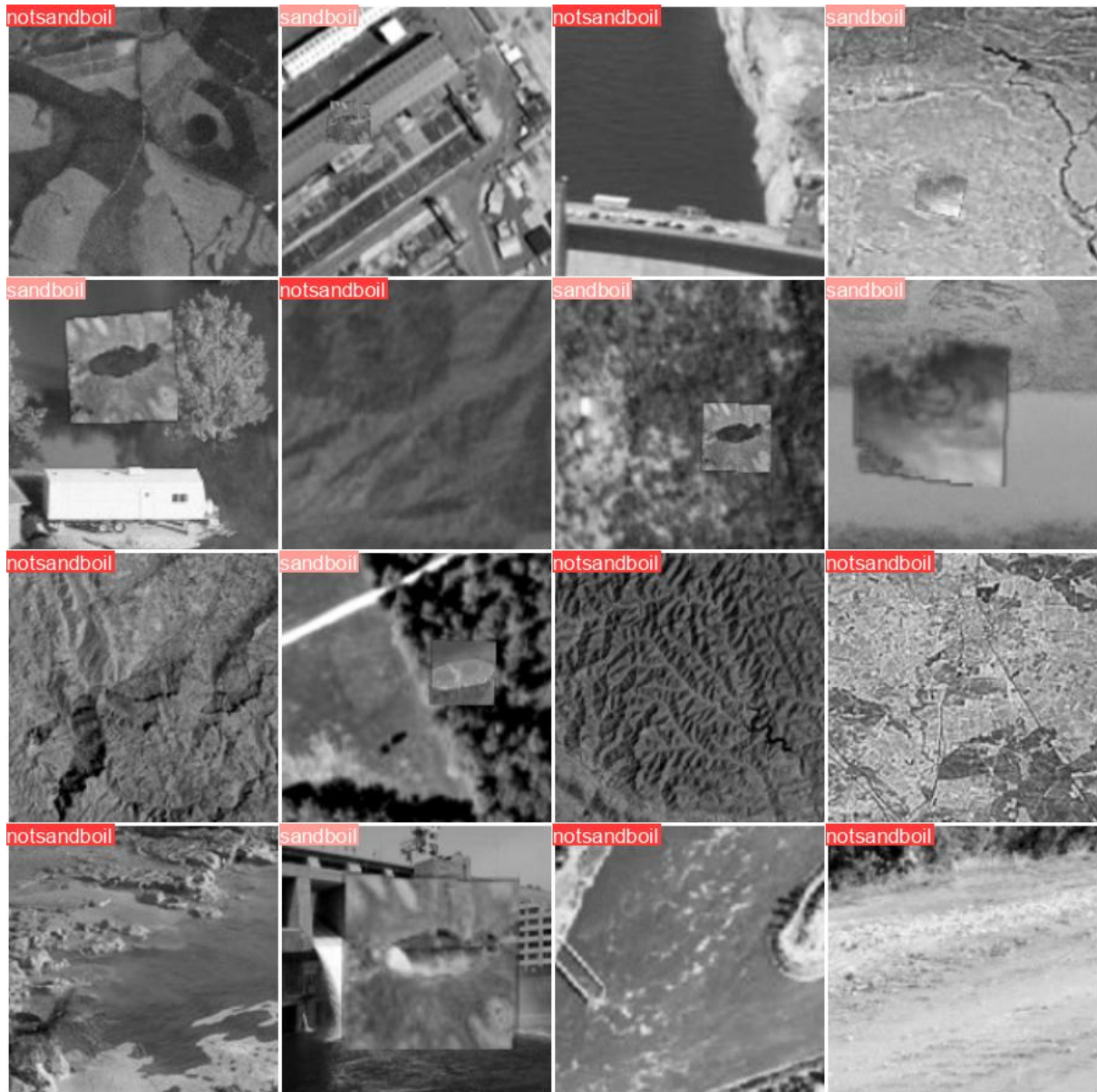


Figure 5.4: YOLO Algorithm Testing Images

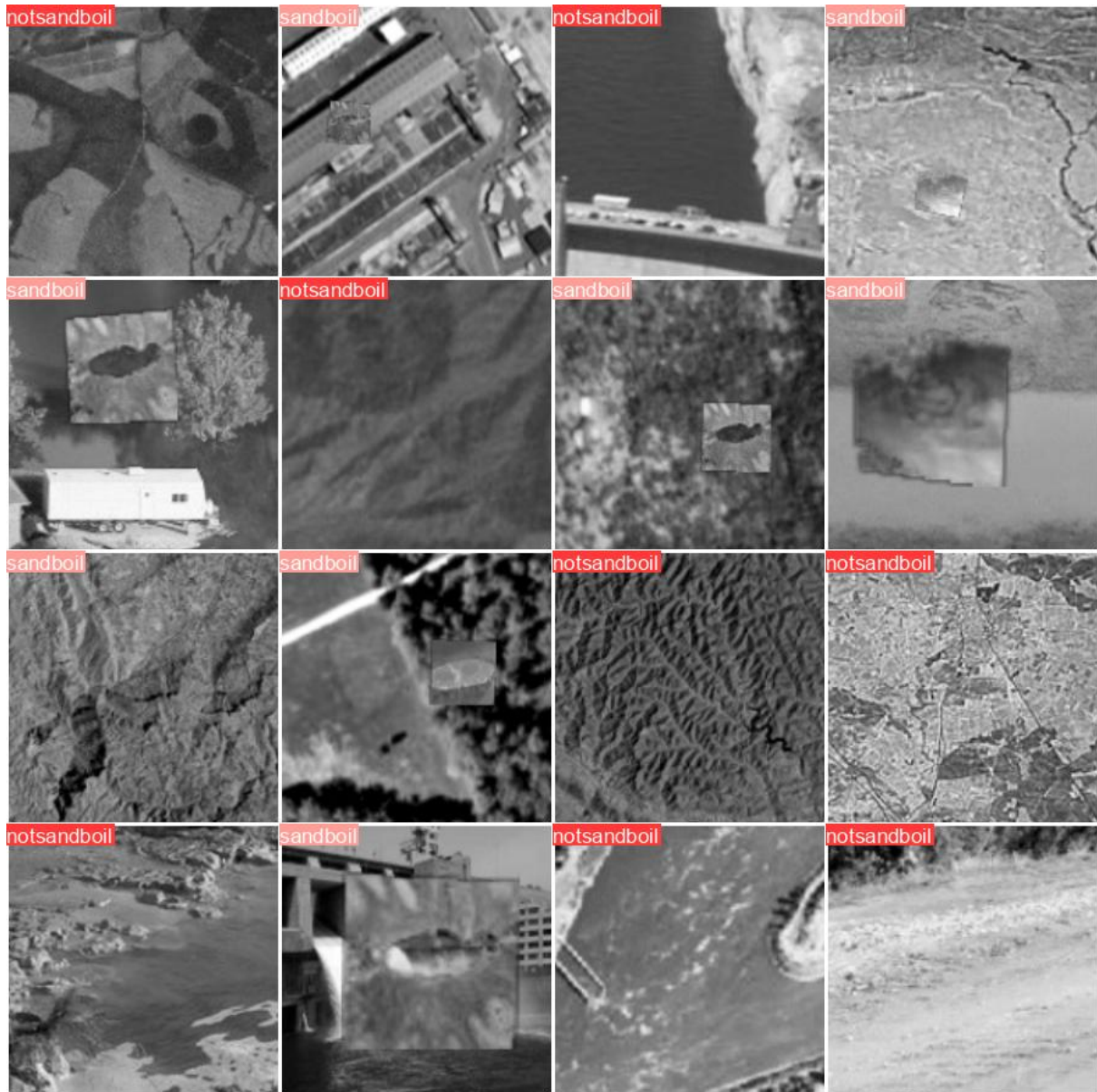


Figure 5.5: YOLO Algorithm Prediction

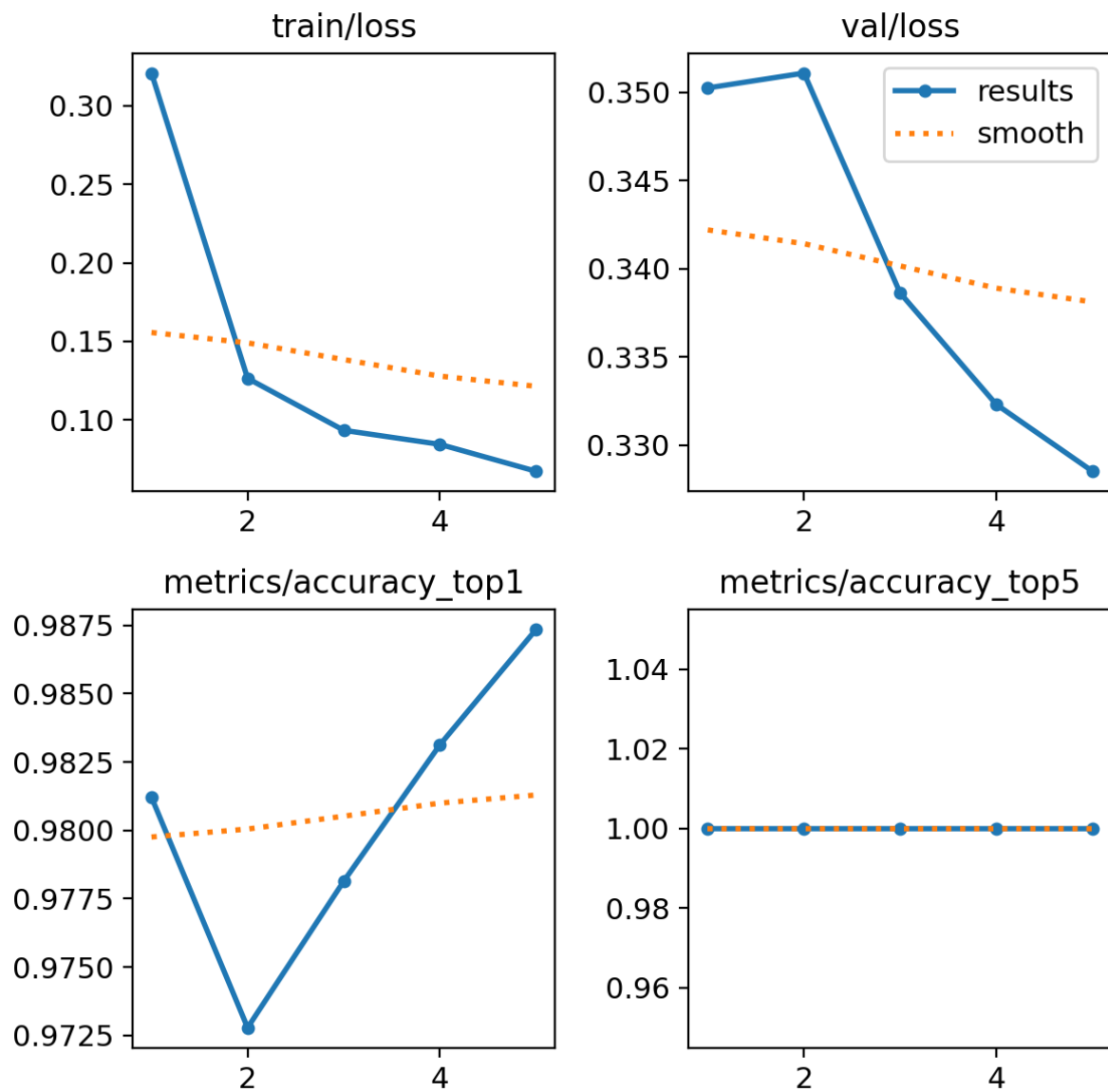


Figure 5.6: YOLO Algorithm Graphs

5.3 SVM

In SVM, our model achieves an impressive overall accuracy of 90.90%, placing it as the second-highest performer among the compared methods. This accuracy indicates the model's ability to correctly classify instances, highlighting its effectiveness in distinguishing sand boils accurately. With a precision of 0.8571 and a recall of 1.0, the SVM demonstrates a strong balance between identifying relevant instances and avoiding false positives. The F1 score, a harmonic mean of precision and recall, stands at 0.9230, further affirming the model's robustness in classification tasks. Leveraging a dataset comprising 249 images for training and 11 images for testing, the SVM model showcases its capacity to generalize well to unseen data, underlining its reliability in real-world scenarios.

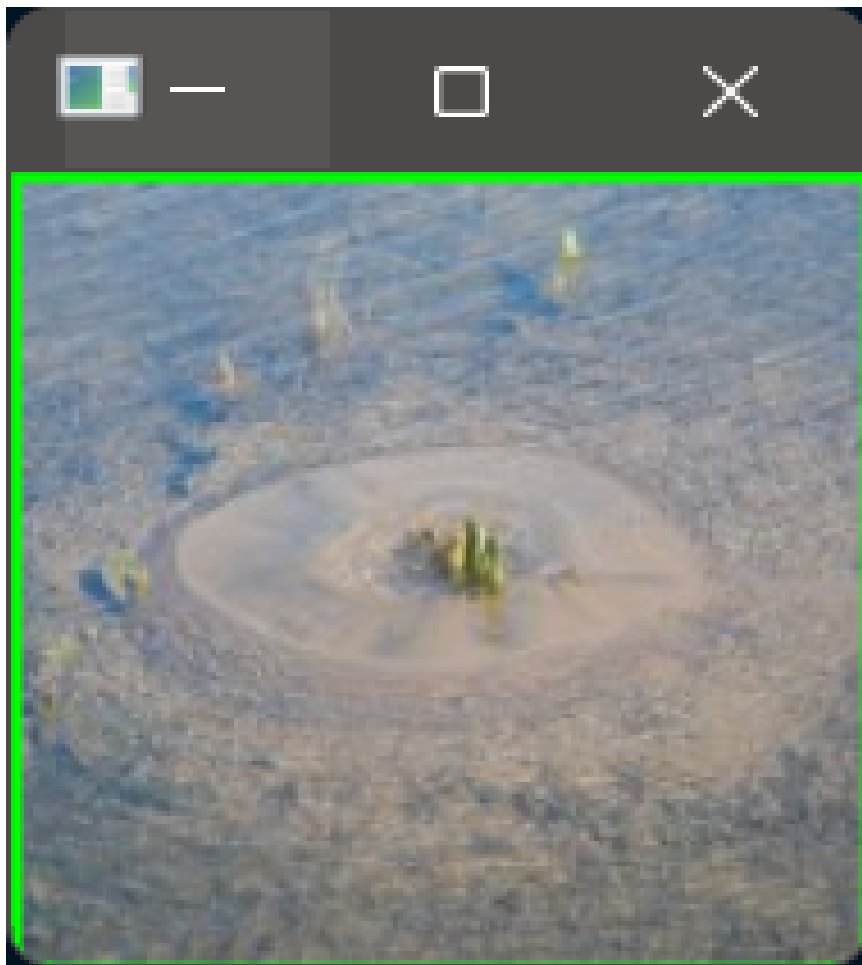


Figure 5.7: SVM Algorithm Detecting Sand Boil

5.4 CNN

In the case of Convolutional Neural Networks (CNN), we attain an overall accuracy of 81.82%, which, comparatively, stands as the lowest among the methods assessed. Despite this, the precision and recall metrics both register at 0.8333, indicating a balanced performance in correctly identifying positive instances while minimizing false positives. The F1 score, a harmonic mean of precision and recall, also aligns at 0.8333, consolidating the model's effectiveness in achieving a balance between precision and recall. Notably, this CNN model was trained on a dataset comprising 249 images and evaluated on a separate test set consisting of 11 images.

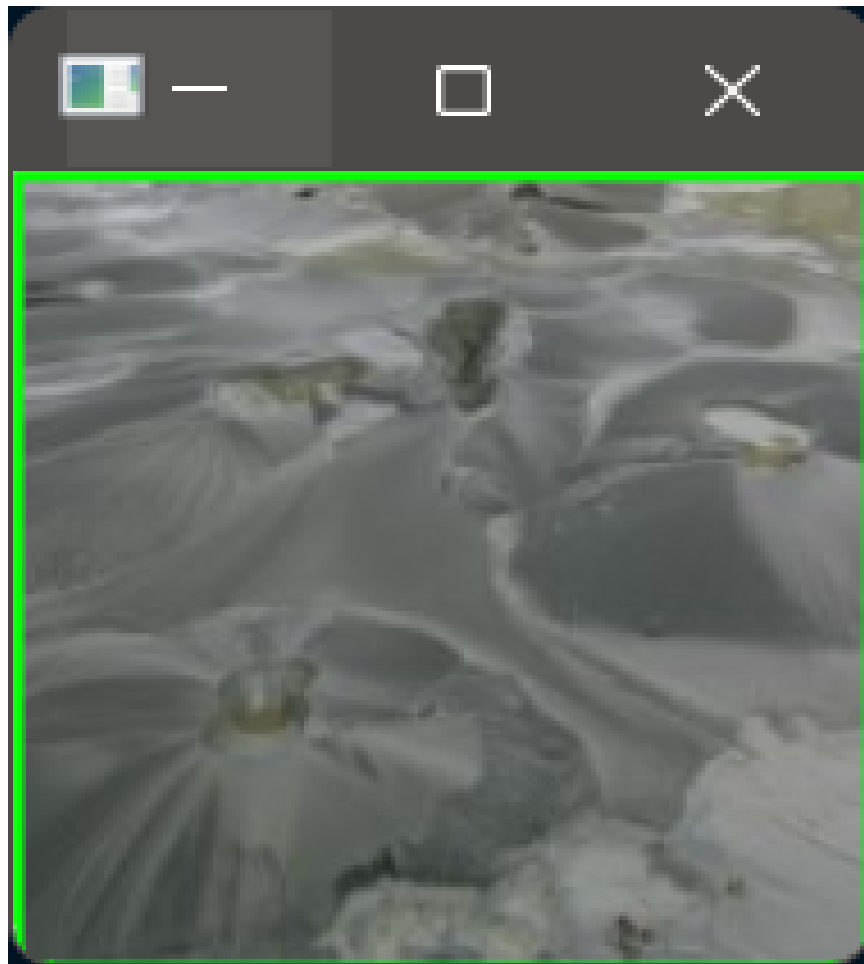


Figure 5.8: CNN Detecting Sand boil

5.5 Comparison Table

After comparing the performance of various object detection algorithms for detecting sand boils, it is evident that each method exhibits distinct strengths and weaknesses. The Viola-Jones algorithm, despite its age, demonstrates commendable performance with an accuracy of 86.11%, underscoring its robustness in object detection tasks. YOLO, on the other hand, emerges as the top performer with an impressive accuracy of 98.74%, showcasing its prowess in accurately detecting sand boils. SVM also proves to be a strong contender with an accuracy of 90.90%, demonstrating its capability in classification tasks. However, CNN lags behind slightly with an accuracy of 81.82%, suggesting room for improvement in its feature extraction capabilities. Overall, while each algorithm presents its unique advantages, YOLO stands out as the most effective solution for detecting sand boils, offering unparalleled accuracy and reliability.

Table 5.1: Comparison of all the Algorithms and the Datasets

Algorithm	Precision	Accuracy	Recall	F1-score	Training	Testing
Viola Jones	86.11%	0.8170	0.8241	0.8205	6305	78
YOLO	98.74%	0.9281	0.9633	0.9453	6000	14
SVM	90.90%	0.8571	1.0	0.9230	249	11
CNN	81.82%	0.8333	0.8333	0.8333	249	11

Chapter 6

Conclusions and Future Work

Here, we compared different object detection methods and determined the best ones to use for the detection of sand boils. We developed a stacking based machine learning predictor which focuses on using the best methods to increase the detection accuracy of the machine learning model.

We also created a database of positive and negative samples of sand boils for use in research. After a comprehensive evaluation of various object detection algorithms for detecting sand boils, it is evident that each method possesses distinct strengths and limitations.

The Viola-Jones algorithm demonstrates commendable performance with an accuracy of 86.11%, underscoring its reliability despite its age. YOLO emerges as the top performer with an impressive accuracy of 98.74%, showcasing its effectiveness in accurately detecting sand boils. SVM also proves to be a strong contender with an accuracy of 90.90%, highlighting its suitability for classification tasks. However, CNN lags behind slightly with an accuracy of 81.82%, indicating potential for improvement in feature extraction capabilities. Overall, YOLO stands out as the most effective solution, offering unparalleled accuracy and reliability in sand boil detection.

For future work, can improve sand boil detection by enhancing existing algorithms like CNNs with advanced architectures or domain-specific knowledge. Additionally, combining algorithms like YOLO and CNN could lead to better performance. Using drones equipped with high-resolution cameras presents a promising opportunity for efficient sand boil detection over

large areas, especially in remote or inaccessible locations. Further research should focus on optimizing algorithms for real-time deployment in diverse environmental conditions. Additionally, augmenting datasets with synthetic data and collecting more diverse samples will improve the accuracy and reliability of sand boil detection algorithms in real-world scenarios.

References

- [1] Ahmed Elgamal Zhaohui Yang. “Sand Boils and Liquefaction-Induced Lateral Deformation”. In: *University of California at San Diego, U.S.A* (2017).
- [2] Chi-Chin Tsai. “Experimental study on the mechanism of sand boils and associated settlements due to soil liquefaction in loose sand”. In: *Engineering Geology, Volume 306* (2022).
- [3] Mitsutoshi Yoshimine Kenji Ishihara. “Evaluation of Settlements in Sand Deposits Following Liquefaction During Earthquakes”. In: *SOILS AND FOUNDATIONS Vol. 32* (1992).
- [4] Thomas Jenkins Amir Jazayeri Adrian D. Werner. “Effects of porous media thickness and its hydraulic gradient history on the formation of sand boils: Experimental investigation”. In: *Elsevier* (1992).
- [5] Andrew John Brennan. *Observations on Sand Boils from Simple Model Tests*. URL: https://www.researchgate.net/publication/269157128_Observations_on_Sand_Boils_from_Simple_Model_Tests. accessed on 16.12.2023.
- [6] Pragati Baheti. *Image Recognition Guide*. URL: <https://www.v7labs.com/blog/image-recognition-guide>. accessed on 15.12.2023.
- [7] Aditi S. Kuchi. “Detection of Sand Boils from Images using Machine Learning Approaches”. In: *University of New Orleans Theses and Dissertations. 2618*. (2019).
- [8] Yi-Qing Wang. “An Analysis of the Viola-Jones Algorithm”. In: *Image Processing On Line (IPOL)* (2014).
- [9] Joseph Redmon Santosh Divvala Ross Girshick Ali Farhadi. “You Only Look Once: Unified, Real-Time Object Detection”. In: *CVPR paper* (2016).
- [10] Wei Liu Dragomir Anguelov Dumitru Erhan Christian Szegedy Scott Reed Cheng-Yang Fu Alexander C. Berg. “SSD: Single Shot MultiBox Detector”. In: *European Conference on Computer Vision* (2016).
- [11] *Machine learning applications in detecting sand boils from images*. URL: <https://www.sciencedirect.com/science/article/pii/S2590005619300128>. accessed on 16.12.2023.